POLITECNICO DI BARI
Department of Electrical and Information Engineering

Master's Degree in
AUTOMATION ENGINEERING

REPORT TECNICO ANNUALE
ROBOTICS – INDUSTRIAL HANDLING

Manipolatore a due gradi di libertà:

scripts' documentation e test di laboratorio.

Professor:
Prof. Ing. Paolo LINO

Students:
Luigi ALBANESE
Nicolò SANTILIO
Luigi SALVI

Academic Year 2024/2025

Sommario

Introduzione 3
1. Hardware del Manipolatore 3
2. Software di Controllo e Interfaccia Utente 3
3. Controllo del Sistema 4
Script principali: documentazione 5
High Level: `main.py` e `trajpy.py` 5
Definizione di parametri e impostazioni 5
Handling delle traiettorie 6
Interazione con il microcontrollore 7
Low Level: `main.c, custom.c e pid_controller.c` 9
Schema dei collegamenti 13
Note 14
Test di laboratorio: valutazioni sul funzionamento 15
FASE 1: Acquisizioni preliminari 15
FASE 2: Setting dei parametri critici 15
FASE 3: Ricalibrazione dei controllori 16
Raccolta dati 21
Dichiarazione delle variabili 21
Estrazione dei dati dai ringbuffer 21
Ottenimento del timestamp 22
Formattazione e stampa dei dati 22
Stampa a video 22
Considerazioni finali e sviluppi futuri 23

Introduzione

Nel presente capitolo viene descritto lo stato dell'arte del manipolatore planare a 2 DoF, evidenziando il lavoro già svolto dal gruppo di lavoro nelle tre macroaree fondamentali: ***hardware, software e controllo***. Sulla base di tale lavoro, il presente elaborato è da intendersi, in modo ambivalente, sia come un'appendice documentativa, in grado di fornire una solida guida all'utilizzo del sistema; sia come un report di validazione, il cui scopo è quello di evidenziare, a valle di una serie di prove su banco, alcune criticità e problematiche che il sistema presenta. Di seguito si riassumono i principali aspetti che caratterizzano il manipolatore.

1. Hardware del Manipolatore

La realizzazione hardware del manipolatore vede l'impiego di varie componenti, alcune delle quali appositamente realizzate, altre direttamente acquistate (elettronica):

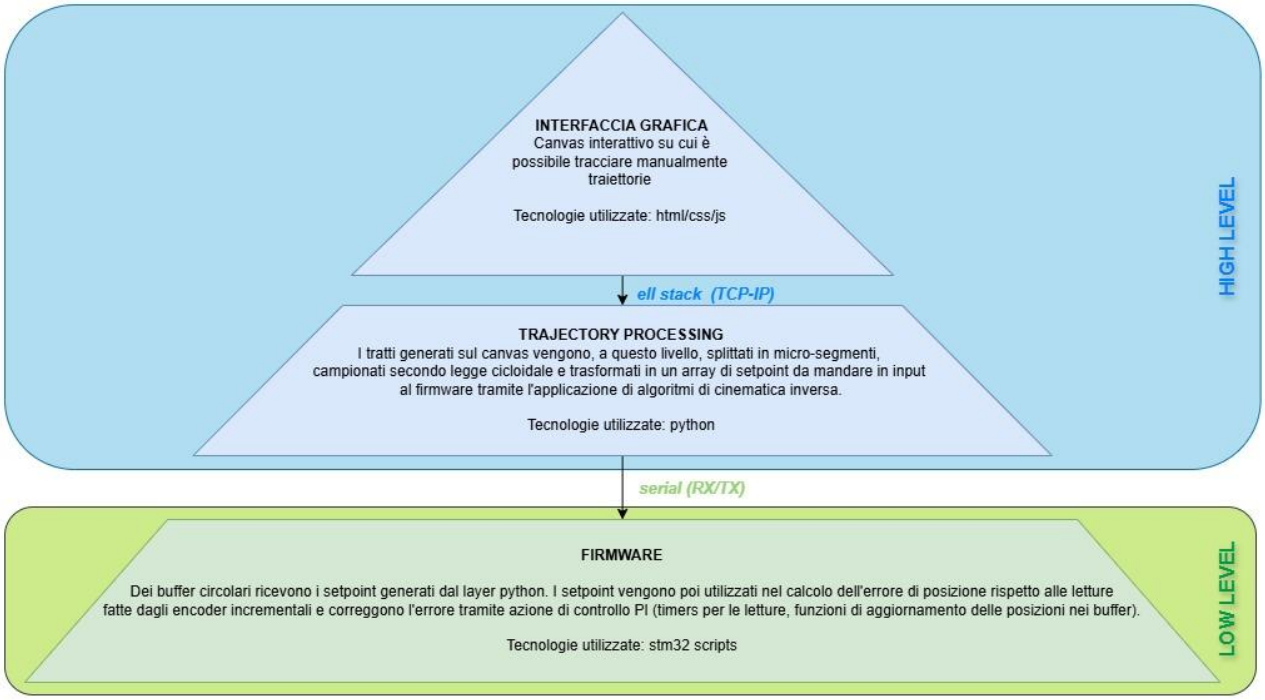
- **Motori ed encoders:** è stato utilizzato un motore passo-passo NEMA 17 per ogni giunto, che monta un encoder magnetico incrementale. Questa soluzione, pur essendo compatta, ha presentato alcune criticità nella gestione della retroazione e sulla generazione di un offset più o meno variabile a regime e non correggibile in anello chiuso.
- **Driver e riduzioni:** il motore è pilotato dal driver TI DRV8825, che permette il microstepping (utilizzato in modalità 1/16, per ottenere fino a 3200 micro-passi per rivoluzione) e garantisce un controllo fine della velocità. Per la trasmissione della coppia, sono stati impiegati riduttori planetari con rapporti di 10:1 e 5:1.
- **Scheda di sviluppo:** il sistema è gestito da una scheda basata su un microcontrollore della famiglia STM32 (STM32F446RE) che offre le risorse necessarie (timers, ADC, interfacce di comunicazione RX/TX) per la gestione delle operazioni di controllo e acquisizione dati in modalità sincrona e asincrona.

2. Software di Controllo e Interfaccia Utente

Lo stack software si articola su due livelli, che interagiscono tra di loro per la gestione delle operazioni compiute dal manipolatore:

- **Software High-Level:** Sviluppato con tecnologie Web (HTML, CSS, JavaScript) e Python (integrato tramite il framework `eel`), questo modulo offre un'interfaccia grafica essenziale, che permette all'utente di disegnare la traiettoria direttamente su un canvas o di scegliere l'esecuzione di traiettorie prefissate. Una volta definita, la traiettoria viene processata – mediante suddivisione in patch campionate con la legge cicloidale – e convertita in una serie di setpoint espressi nello spazio dei giunti, grazie alla computazione della cinematica inversa. Tutte queste operazioni avvengono in asincrono: il manipolatore si muove solamente dopo che gli array di setpoint di traiettoria vengono generati ed inviati per intero.
- **Software Low-Level (Firmware):** Il firmware, eseguito sul microcontrollore, gestisce l'intero ciclo di controllo in tempo reale. Tra le sue funzioni principali si annoverano:
    - **Controllo PID:** Implementazione dei controllori PID (o PI, a seconda della modalità operativa), con strutture dati dedicate che gestiscono le variabili di memoria e il calcolo dell'azione di controllo, con aggiornamenti periodici gestiti tramite timer hardware.
    - **Gestione delle comunicazioni:** Utilizzo del protocollo USART in modalità DMA per lo scambio di dati (setpoint e letture dagli encoder) tra il PC e il microcontrollore.

- **Strutture di Supporto:** Implementazione di strutture dati come il ringbuffer, essenziali per la gestione FIFO dei dati ricevuti.



3. Controllo del Sistema

Si è scelto di implementare una tecnica di controllo (decentralizzato) di posizione nello spazio dei giunti, che prevede l'utilizzo di due controllori PID indipendenti, uno per ciascun giunto del manipolatore.

- **Acquisizione e generazione delle traiettorie:** la gestione della traiettoria si basa su un'interfaccia utente (high-level sw.) che consente di disegnare direttamente sulla GUI la traiettoria desiderata, mediante primitive geometriche (linee e circonferenze). In alternativa è possibile selezionare due diverse operazioni da sottoporre al manipolatore: la procedura di Homing (necessaria per il setting degli encoders incrementali) e la traiettoria ripetibile (repeatable trajectory), ovvero una traiettoria predefinita. Queste traiettorie vengono quindi suddivise in segmenti tramite legge cicloidale e quindi campionati in una serie di punti. Successivamente, l'algoritmo di cinematica inversa converte i setpoint dallo spazio operativo allo spazio dei giunti.

- **Implementazione del controllo in anello chiuso:** per garantire un movimento pulito e compensare eventuali errori è stato implementato un controllo in anello chiuso. Il feedback viene acquisito tramite gli encoder incrementali e, grazie alla lettura della posizione del motore, viene calcolata l'azione di controllo PID che genera un comando di velocità, il quale viene convertito in un segnale PWM per pilotare i driver dei motori. È bene specificare che le grandezze di posizione retroazionate nel control-loop non sono le letture dirette degli encoder, ma delle quantità opportunamente elaborate, al fine di considerare i meccanismi di riduzione montati sul manipolatore (una volta settato opportunamente l'auto-reload register dei timer che puntano ai motori, il fattore di riduzione o la frequenza di clock sono parametri già considerati a monte dell'elaborazione dati).

Script principali: documentazione

Dal momento che non è disponibile una documentazione esaustiva degli script presenti nel progetto, mantenendo viva la suddivisione in high e low level, procediamo con un'analisi documentativa degli script di maggior rilievo su cui si fonda il funzionamento dell'intero stack.

High Level: `main.py` e `trajpy.py`

Lo script `main.py` funge da amministratore del sistema, gestendo l'acquisizione dei dati dalla GUI, il calcolo delle traiettorie e l'invio delle informazioni al microcontrollore tramite comunicazione seriale.

Per realizzare queste operazioni, `main.py` si interfaccia con la libreria custom `trajpy.py`, che fornisce funzioni per la generazione delle traiettorie e per la conversione tra coordinate cartesiane e coordinate dei giunti. *È doveroso sottolineare che molte delle funzioni contenute all'interno di questa libreria risultano inutilizzate e sono state probabilmente concepite per scopi di testo e/o debug.*

Al fine di una trattazione organica e funzionale, analizziamo nel seguito il comportamento del main script e, sulla base delle funzioni che questo chiama dalla libreria custom, ne descriviamo il comportamento di ognuna.

Lo script `main.py` è organizzato in tre macro-sezioni:

Definizione di parametri e impostazioni

Vengono istanziate ed inizializzate costanti e funzioni necessarie alla gestione delle traiettorie. In particolare, nella parte iniziale dello script vengono avvalorati una serie di parametri e certamente di fondamentale importanza nella generazione delle traiettorie sono quelli contenuti all'interno dell'array `settings`:

```
settings = {
    'Tc' : 0.01, # s
    'data_rate': 1/100, # rate at which msgs are sent
    'max_acc' : 0.35,#0.1,#1.05, #1.05, # rad/s**2
    'ser_started': False,
    'line_tl': lambda t, tf: tpy.cycloidal([0, 1], 2, tf)[0][0](t),
    'circle_tl': lambda t, tf: tpy.cycloidal([0, 1], 2, tf)[0][0](t)
}
```

Dove:

- `Tc` rappresenta il tempo di campionamento della traiettoria (non per forza uguale al tempo di campionamento del microcontrollore, dettaglio a cui prestare attenzione in fase di testing);
- `data_rate` è la velocità con cui vengono inviati i dati tramite seriale. Il parametro settato in origine (in figura) si è dimostrato, in fase di test, errato: infatti, se scelto troppo alto, causa una disparità nella velocità di scrittura e lettura dei buffer circolari lato firmware che porta al funzionamento scorretto del manipolatore. È stato individuato come parametro ottimale il valore 1/1000.
- `max_acc` è la massima accelerazione raggiungibile dal manipolatore. Questo parametro viene poi utilizzato nella generazione dei setpoints, quando la traiettoria complessiva viene suddivisa in un insieme di segmenti lineari o circolari: qui il massimo valore di accelerazione è utile, nota la lunghezza del tratto, per il calcolo del tempo di esecuzione dello stesso.

  ```
  tf = sqrt(2*pi*length/kargs['max_acc'])
  ```
- `line_tl`, `circle_tl` consentono di istanziare, per ogni tipologia di segmento di traiettoria (sia esso lineare o circolare) la tipologia di legge temporale. Nel manipolatore in oggetto è stata

scelta per entrambi una legge di tipo cicloidale, come già accennato nell'introduzione, implementata da un'apposita funzione contenuta nella libreria `trajpy.py`.

Handling delle traiettorie

In questa parte del codice vengono processati i dati della GUI e generate le traiettorie. Qui, le funzioni di maggior rilievo sono `py_get_data, slice_trj` e `trace_trajectory`.

L'obbiettivo di `py_get_data` è quello di acquisire i dati della traiettoria tracciata sul canvas e di generare delle traiettorie composte da coordinate nello spazio dei giunti, che diventeranno i setpoint da inseguire in anello chiuso. Si tratta della funzione principale all'interno del main e per questa ragione troviamo, al suo interno, chiamate ad altre funzioni, sia contenute nel main, sia appartenenti alla libreria `trajpy.py`.

Questa funzione `py_get_data`, in ordine:

1. Recupera i dati dei punti di traiettoria dalla GUI;
2. Calcola i movimenti del manipolatore con `slice_trj`, suddividendo il percorso in segmenti più piccoli e genera così una serie di punti (nello spazio dei giunti) corrispondenti alla traiettoria da seguire;
3. Deriva numericamente le posizioni dei giunti per ottenere i profili di velocità (`find_velocities`) e accelerazione (`find_accelerations`);
4. Invia la traiettoria calcolata al microcontrollore con `send_data`;
5. Traccia la traiettoria sul canvas con `trace_trajectory`. Ci si aspetterebbe che la traiettoria tracciata sul canvas non sia quella precalcolata di riferimento, ma quella realmente inseguita e prelevata dai dati degli encoder, ma il progetto non è attualmente predisposto per questa feature.

Gran parte delle attività svolte da `py_get_data` riguardano l'elaborazione matematica della traiettoria, performata dalla funzione `slice_trj`, presa dalla libreria custom `trajpy.py`.

In effetti, la funzione `slice_trj` è responsabile della suddivisione di una traiettoria in segmenti più piccoli, applicando una legge temporale passata in input (`line_tl,circle_tl`) per calcolare i punti intermedi. Inoltre, converte le coordinate cartesiane in coordinate di giunto (applicando una funzione di cinematica inversa, nel codice indicata come `ik`) per generare la traiettoria del manipolatore.

Questa funzione `slice_trj`, in ordine, si occupa di:

1. Specificare una politica di gestione dei parametri mancanti, specificando alcuni valori di default qualora i parametri in input (`settings`) non siano avvalorati correttamente;
2. Per ogni segmento, estrarre i punti di inizio e fine, che vengono convertiti in oggetti di classe `Point`. A conversione avvenuta è poi possibile calcolare la lunghezza del tratto: se il segmento è lineare, viene calcolata la distanza euclidea tra il punto di inizio e di fine; se invece il segmento è circolare, vengono calcolati raggio e angolo di rotazione, partendo dal centro della circonferenza e dai punti di inizio e fine, forniti in input;
3. Calcolare il tempo di esecuzione di ogni segmento di traiettoria `tf` (come visto in precedenza);
4. Gestire la traiettoria in caso di valore del parametro `penup=False`, dove il manipolatore è in fase di scrittura ed è richiesta una diminuzione dell'accelerazione massima ed un calcolo separato delle traiettorie di giunto.
5. Generare le traiettorie effettive, a seconda che si tratti di segmenti lineari o circolari;
6. Convertire i punti appartenenti alle traiettorie generate in coordinate di giunto tramite la funzione di cinematica inversa `ik`;

In output, questa funzione fornisce `q0s, q1s, penups, ts`: i primi due valori altro rappresentano le coordinate di giunto della traiettoria di riferimento, la lista `penups` contiene una serie di flag binari che indicano se in un certo segmento di traiettoria il manipolatore è in scrittura o meno; `ts` è invece una lista dei sequenziali timestamps per ognuna delle pose del manipolatore durante la traiettoria di riferimento.

Infine, l'ultima funzione richiamata in get_py_data è trace_trajectory. Questa funzione si occupa di plottare la traiettoria calcolata agli step precedenti sulla GUI. In sostanza, riceve le coordinate di giunto nel tempo, le converte in coordinate spaziali mediante l'applicazione di cinematica diretta (dk) e genera il plot della traiettoria di riferimento. È importante specificare che, essendo la traiettoria di riferimento direttamente tracciata sul canvas del manipolatore, può indurre l'utente a scambiarla con il reale percorso spazzato dal manipolatore. Sarebbe più opportuno plottare in quella sede il reale percorso eseguito dal manipolatore, preso dai dati registrati dai due encoders. Per far ciò sarebbe quindi necessario costruire un tipo di comunicazione sincrona e valutare la presenza di più bus di comunicazione e non solamente uno, come si presenta allo stato dell'arte.

Interazione con il microcontrollore

Questa macro-sezione del codice principale è dedicata all'invio delle traiettorie via seriale, gestendo la chiusura sicura della comunicazione una volta terminate le operazioni di interpolazione della traiettoria.

La comunicazione in seriale viene aperta all'interno del main e, una volta che il codice genera le traiettorie da inviare al microcontrollore, l'invio tramite seriale viene gestito interamente dalla funzione `send_data`, richiamata come già anticipato dal `get_py_data`.

La funzione `send_data` provvede a convertite in esadecimale tutte le quantità da inviare a mezzo seriale, scrive l'array dei dati strutturati sulla seriale e regola la velocità di invio dei dati utilizzando come rate il parametro `data_rate` istanziato in `settings`.

La struttura dati che viene scritta su seriale è nella seguente forma:

```
msg_str = f"TRJ:{d2h(data['q'][0][i])}:{d2h(data['q'][1][i])}"+\
          f":{d2h(data['dq'][0][i])}:{d2h(data['dq'][1][i])}"+\
          f":{d2h(data['ddq'][0][i])}:{d2h(data['ddq'][1][i])}"+\
          f":{int(data['q'][2][i])}\n"
```

Posizione
Velocità
Accelerazione
Penup Flag

L'array `['q']` è composto da 3 elementi: `['q'][0]` e `['q'][1]` sono rispettivamente le variabili di giunto del primo e del secondo motore, `['q'][2]` è il flag di `penups` che indica se, per una certa coppia di `q0` e `q1`, la penna è sollevata o no. Inoltre, il flag `TRJ` posto all'inizio di ogni stringa di esadecimali inviata è indicativo del fatto che il front-end sta inviando al controllore un oggetto di tipo traiettoria. Qualora il controllore ricevesse un oggetto con il flag `HOM`, allora, lato firmware, viene gestita la procedura di homing che provvede a settare il manipolatore in modo tale che gli encoder incrementali siano pronti alla lettura.

Funzione	Descrizione	Usata?
`print_error`	Stampa errori su stderr.	Sì
`handle_closure`	Chiude la comunicazione seriale.	Sì
`py_get_data`	Acquisisce dati dalla GUI e genera la traiettoria.	Sì
`send_data`	Invia la traiettoria al microcontrollore via seriale.	Sì
`trace_trajectory`	Visualizza la traiettoria generata sulla GUI.	Sì
`debug_plot`	Genera e salva un grafico di una traiettoria.	No
`debug_plotXY`	Genera e salva un grafico XY della traiettoria.	No
`d2h`	Converte un numero float in esadecimale.	Sì
`h2d`	Converte un numero esadecimale in float.	No
`py_log`	Stampa un messaggio di log.	No
`py_log_data`	Salva i dati della traiettoria in un file CSV.	No
`py_homing_cmd`	Invia il comando di homing al microcontrollore.	Sì
`py_serial_online`	Verifica se la seriale è attiva.	Sì
`py_serial_startup`	Inizializza la comunicazione seriale.	Sì

Tabella 1: Funzioni main.py

Funzione	Descrizione	Usata?
`ew`	Calcola il prodotto scalare	No
`angle_between`	Calcola l'angolo tra due vettori.	No
`time_row`	Genera una riga della matrice di Vandermonde.	No
`spline3`	Calcola i coefficienti di una spline cubica.	No
`spline5`	Calcola i coefficienti di una spline di 5° ordine.	No
`rangef`	Genera una lista di punti intermedi con step personalizzato.	Sì
`compose_spline3`	Costruisce una traiettoria con spline cubiche.	No
`cubic_speeds`	Calcola le velocità intermedie tra punti di una spline.	No
`slice_trj`	Suddivide una traiettoria in punti intermedi e applica IK.	Sì
`find_velocities`	Calcola la velocità istantanea della traiettoria.	Sì
`find_accelerations`	Calcola l'accelerazione istantanea della traiettoria.	Sì
`dk`	Calcola la cinematica diretta (giunti → cartesiano).	Sì
`ik`	Calcola la cinematica inversa (cartesiano → giunti).	Sì
`cycloidal`	Genera una traiettoria cicloidale.	Sì
`compose_cycloidal`	Costruisce una traiettoria con più segmenti cicloidali.	No
`trapezoidal`	Genera una traiettoria con profilo trapezoidale.	No
`compose_trapezoidal`	Costruisce una traiettoria con profilo trapezoidale.	No

Tabella 2: funzioni trajpy.py

Low Level: `main.c, custom.c e pid_controller.c`

Il livello di bassa programmazione dello stack comprende tutti gli script di progettazione del firmware. Per la generazione del progetto firmware è stato utilizzato l'ambiente di progettazione CubeIDE fornito da STM, che genera automaticamente la project-folder con tutti i file di configurazione e dipendenze preimpostate. I file che rappresentano il cuore implementativo e che sono stati modificati appositamente per l'interfacciamento con il front-end e con il manipolatore sono essenzialmente tre:

- `main.c` è il punto di ingresso principale del programma e governa l'esecuzione complessiva del sistema;
- `custom.c` contiene funzioni di gestione del manipolatore e l'implementazione della legge di controllo basata sulla dinamica inversa;
- `pid_controller.c` implementa un controllo PID generico utilizzato per il controllo di posizione (e predisposto per il controllo di velocità, ma non utilizzato nell'esecuzione delle traiettorie).

Nel seguito, come fatto per l'high-level, procediamo con la descrizione sistematica di ognuno dei componenti del firmware, al fine di comprenderne il funzionamento e fornire una documentazione solida e di facile consultazione.

Il file `main.c` rappresenta il cuore gestionale del firmware del sistema embedded: esegue le operazioni di inizializzazione e gestione del controllo del manipolatore robotico. Questo script utilizza la HAL (Hardware Abstraction Layer), un set di routine che tradizionalmente utilizza l'ambiente STM per gestire le periferiche, come:

- **Timer (TIM2, TIM3, TIM4, TIM5, TIM10)** per il controllo della PWM e la lettura degli encoder incrementali montati su ciascun motore;
- **UART2** per la comunicazione seriale tra front-end e microcontrollore;
- **DMA** per la gestione dei dati ricevuti e trasmessi.

In ordine logico di compilazione, lo script `main.c` si occupa di:

1. *Inizializzare e configurare le periferiche hardware*. Dapprima avviene il reset dei registri e il setup del clock (`HAL_Init()`, `SystemClock_Config()`); successivamente si procede con l'inizializzazione delle periferiche hardware.

`MX_GPIO_Init`	Inizializza i GPIO pins (opportunamente configurati) per l'input, l'output e gli interrupts.
`MX_DMA_Init`	Attiva il Direct Memory Access
`MX_USART2_UART_Init`	Inizializza l'USART2 (interfaccia seriale) per la comunicazione UART (opportunamente configurata).
`MX_TIM3_Init`	Inizializza il timer TIM3 in modalità encoder, poiché pensato per leggere i valori raccolti dall'encoder del motore 1 (primo giunto)
`MX_TIM4_Init`	Inizializza il timer TIM4 in modalità encoder, poiché pensato per leggere i valori raccolti dall'encoder del motore 2 (secondo giunto)
`MX_TIM2_Init`	Inizializza il timer TIM2 in modalità PWM (opportunamente configurato).
`MX_TIM5_Init`	Inizializza il timer TIM5 in modalità PWM (opportunamente configurato).
`MX_TIM10_Init`	Inizializza il timer TIM10 come contatore (base timer).

Tabella 3: Periferiche hardware

2. *Inizializzare e configurare il manipolatore e i controllori (PI)*. Ricorrendo a delle funzioni di inizializzazione definite in `custom.c` e `PID_controller.c`, gli oggetti *manipolatore* e *controllori* vengono inizializzati, passando gli opportuni timer e parametri in input (`init_man`, `PID_init`). A seguire, viene inizializzato anche il canale di ricezione dati UART con DMA.

3. *Avviare e configurare gli encoder prima di entrare nel ciclo principale*. Tramite le funzioni `start_timers` e `setup_encoders`, vengono attivati i timer per il controllo della PWM e per le letture dagli encoders.
4. *Avviare l'infinite loop che gestisce l'intero ciclo di controllo del manipolatore durante il suo funzionamento*. Il ciclo è composto da vari step, alcuni dei quali opzionali a seconda della scelta fatta dall'utente in fase di input. In particolare, molte delle funzioni definite in `custom.c` vengono richiamate:
    - Se l'utente sceglie di eseguire la procedura di setting degli encoders (procedura di homing), il ciclo entra in una condizione *if* e viene chiamata, dal file `custom.c` la funzione `homing()`, che esegue il riposizionamento del manipolatore in posa di partenza, tramite un controllo congiunto di posizione e velocità (a valle della procedura, vengono aggiornate, in ogni step, le velocità tramite `update_speeds()`).
    - Se l'utente sceglie di far eseguire al manipolatore una qualsiasi traiettoria in input, allora vengono chiamate, in sequenza, le funzioni `PID_controller_position()` e `apply_velocity_input()`, le quali si occupano, rispettivamente di calcolare l'azione di controllo PI (non PID, componente D imposta sempre a zero, e quindi inutilizzata) sull'errore di posizione e di applicarla sotto forma di input di velocità ai due motori. Per il calcolo dell'errore di posizione, vengono prelevate le letture degli encoders e scritte su dei buffer circolari a disposizione delle funzioni che implementano il PID. Chiaramente, l'esecuzione di queste due funzioni, come vedremo, sottintendono l'utilizzo dei timers inizializzati per la gestione degli encoders e degli impulsi PWM.
    - L'intero ciclo while è pensato per iterare indefinitamente ed ogni iterazione ha una cadenza temporale personalizzabile e funzione del tempo di campionamento Tc: nello specifico ogni step del ciclo viene eseguito con un ritardo di almeno 1000 volte il tempo di campionamento. Questa dinamica è gestita dalla funzione `rate_spleep()`. Quando il loop viene interrotto (manualmente), vengono messi in stato di stop tutti i timers avviati (sia quelli hardware, che il contatore).
    - *È prevista anche una funzione che permette di monitorare le letture di alcuni dati durante l'esecuzione di una traiettoria, `log_data()`. Questa funzione contenuta nel loop, allo stato dell'arte del progetto non viene mai chiamata e, se forzata la sua chiamata, non produce dati utili. È stato compito di chi redige il presente report modificarla, rendendola funzionante e funzionale al prelievo dati per i test di laboratorio. Successivamente, si tratterà nel dettaglio tale modifica.*

A questo punto, descritto il flusso operativo che ha luogo quando il main del firmware viene eseguito, possiamo passare all'analisi dettagliata delle funzioni che vengono coinvolte, direttamente e indirettamente, nell'esecuzione del ciclo while appena descritto. Come anticipato, gli script che contengono le funzioni utilizzate dal firmware per performare il movimento e il controllo sono `custom.c` e `PID_controller.c`, due librerie personalizzate che analizziamo nel seguito.

Volendo suddividere il contenuto del `custom.c` a seconda del tipo di azioni performate dalle varie funzioni, possiamo identificare diverse sezioni tematiche:

1. *Gestione degli interrupt*: questa categoria di funzioni si occupa di fornire le logiche di gestione ed elaborazione dati in caso di interrupt provenienti da periferiche hardware (encoders, GPIO) e software (seriale).

`HAL_GPIO_EXTI_Callback`	Questa funzione viene chiamata a seguito di un GPIO interrupt: considera quando i limit switch di ogni motore vengono attivati e aggiorna lo stato del manipolatore, considerando l'offset se è in corso l'homing.
`HAL_TIM_PeriodEnlapsedCallback`	Questa funzione viene chiamata ogni qualvolta trascorre un *timer period*: regola la lettura degli encoders ad intervalli regolari.
`HAL_UART_RxCpltCallback`	Questa funzione viene chiamata all'arrivo di un interrupt UART (ricezione dati avvenuta): processa i dati, analizza gli header-flag ('`TRJ`' se arriva una traiettoria di riferimento, '`HOM`' se è in corso l'homing) e aggiorna lo stato del manipolatore riempendo opportunamente i ring-buffers. (*)

Tabella 4: Interrupt Callback functions

(*) Viene menzionato un terzo flag '`POS`', che sarebbe stato pensato per l'attivazione della funzione `Log_data`, ma che non viene mai utilizzato.

2. *Inizializzazione del manipolatore*: la funzione già menzionata `init_man()`, prendendo in input il manipolatore (oggetto `man_t`) e i puntatori ai timer utilizzati per la lettura degli encoder e per la PWM dei motori, provvede ad inizializzare tutti i ring buffers e a settare i timers associati agli encoders e ai motori.
3. *Calcolo delle matrici dinamiche e definizione operazioni matriciali*: le funzioni `B_calc` e `C_calc` si occupano del calcolo, secondo la definizione teorica del modello a dinamica inversa (semplificato), delle matrici di inerzia $B(q)$ e di Coriolis $C(q,\dot{q})$. In questa sezione rientrano anche una serie di funzioni analitiche create per poter performare semplici operazioni come la somma o il prodotto tra matrici, il determinante, l'inversa e la pseudo-inversa di una matrice, e diverse altre di seguito.
4. *Legge di controllo a dinamica inversa*: la funzione `controller` implementa la legge di controllo a dinamica inversa così come presentata nella teoria classica, esplicitando la relazione base rispetto alla velocità (per ogni giunto) di modo che sia possibile passarla come input ai motori, che accettano velocità come grandezze in ingresso. La legge implementata è:
$$B(q)\ddot{q} + C(q,\dot{q})\dot{q} = \tau \;\rightarrow\; \dot{q} = C(q,\dot{q})^{-1} \cdot (\tau - B(q)\ddot{q})$$

5. *Lettura e gestione degli encoder*: grazie alla funzione `read_encoders`, il sistema è in grado, nell'ordine, di prelevare i dati grezzi inviati dagli encoder tramite i timers ad essi associati, convertire tali valori in posizioni angolari (variabili di giunto) considerando la conversione in radianti, il fattore di riduzione e i valori dei registri dei timers associati alle letture di quei valori (ARR- Auto Reload Register) ed infine scrive sui ring-buffer (`q0_actual`; `q1_actual`) le posizioni aggiornate rilevate.
6. *Gestione dei motori*: le funzioni che generano i segnali PWM che pilotano i motori controllati sono `apply_velocity_input` ed `apply_position_input`. La prima viene utilizzata esclusivamente nella procedura di homing, dove è applicato un controllo in velocità. Applica le velocità desiderate tramite segnali PWM, formalizzati calcolando i valori di ARR e CRR (registri dei timer) basati sulle velocità desiderate in input. La seconda, utilizzata invece per le normali procedure di inseguimento della traiettoria, applica le posizioni desiderate ai motori convertendole in input di velocità (essendo questa la grandezza di controllo accettata). Viene utilizzata la precedentemente menzionata legge cicloidale per calcolare la durata del tratto

percorso dal manipolatore rispetto alla massima accelerazione consentita (utilizzati valori diversi di accelerazione massima a seconda dell'ampiezza del movimento registrato). Una volta in possesso del tempo di percorrenza, è possibile esprimere l'input di posizione come input di velocità dividendo lo spostamento effettuato (posizione desiderata – posizione corrente) per il tempo appena calcolato. Una volta convertiti gli input di posizione in input di velocità, come nel caso della funzione `apply_velocity_input`, si calcolano i valori dei registri relativi a quelle velocità, in grado di generare il segnale PWM necessario.

In questa categoria di funzioni rientrano anche `PID_controller_position` e `PID_controller_velocity`, che seguono essenzialmente il medesimo schema visto per le altre due funzioni di input calcolando i valori delle velocità di controllo generate dall'applicazione dei PID sul sistema. Le chiamate a queste funzioni avvengono principalmente all'interno dell'infinite loop del main e sostanziano l'applicazione effettiva del controllo PID.

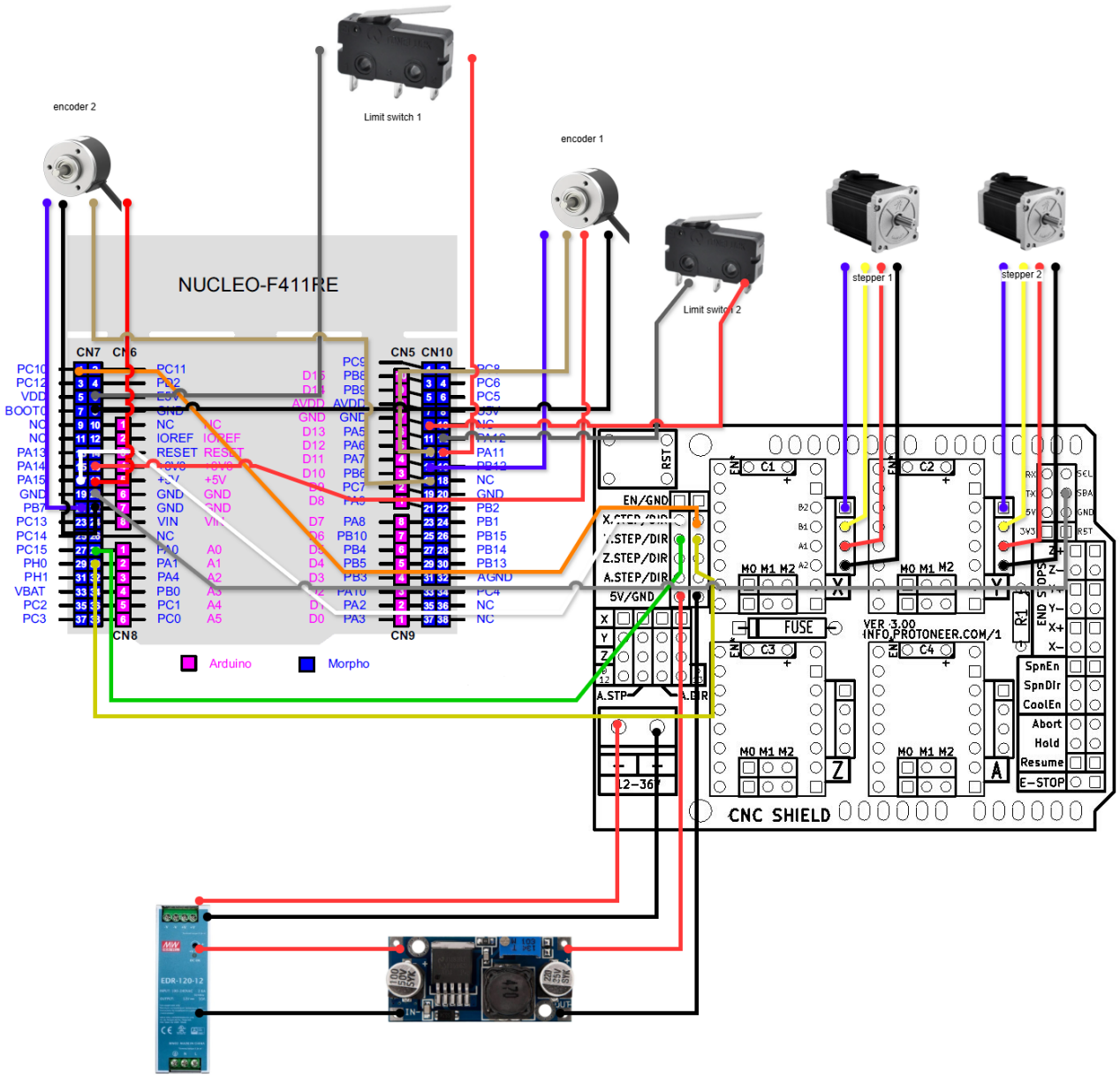
7. *Homing del manipolatore*: alla ricezione del flag 'HOM' da seriale, avvia la sequenza di homing del manipolatore, che consiste nel far muovere i motori verso un punto home predefinito (non è chiaro come siano state stimate le posizioni di questo punto e probabilmente gli errori di posizione a regime non nulli potrebbero dipendere da questo) usando un controllo di velocità prima, aggiornando gli offset registrati poi, ed infine ricorrendo ad un controllo di posizione per raggiungere la posizione home desiderata correggendo gli offset.

Infine, lo script `pid_controller.c`, costituito da solo due funzioni base, consente di inizializzare il controllore sulla base dei parametri necessari (costanti di controllo) e di implementare matematicamente la legge di controllo PID canonica. Le due funzioni vengono, come già anticipato, richiamate all'interno delle funzioni `PID_controller_position` e `PID_controller_velocity` che consentono di generare direttamente l'ingresso di controllo. È giusto specificare che, per scelte di progettazione, la costante Td associata alla componente derivativa del controllore è posta uguale a zero. Il modo in cui questa assegnazione è stata realizzata impedisce, però, di creare un PID ed implementerà, in qualsiasi caso, un PI, anche se l'utente sceglie di assegnare Td≠0 oppure sceglie l'opzione 'PID' settando opportunamente un flag di input alle funzioni appena descritte.

```c
/* output  */
if (pid->type>0){
    u=proportional+pid->integrator;
}else{
    /*derivative contribute*/
    pid->derivative= (2*(pid->Kp)*alpha*error - pid->derivative*(1-(2*alpha)/pid->N))/(1+(2*alpha)/pid->N);
    u=proportional+pid->integrator+0*pid->derivative;
}
```

Nello snippet fornito, si nota che, se l'utente dovesse mandare in input un PID.type=1, si entrerebbe nell'else, ma verrebbe ugualmente implementato un controllore PI, poiché la componente derivativa è sempre moltiplicata per zero (e non per Td, come ci si aspetterebbe).

Schema dei collegamenti



Dispositivo	Pin Dispositivo	Pin su Nucleo-F411RE	Pin su CNC Shield	Colore Cavo
Alimentatore Step-down	OUT+	5V	-	Rosso
Alimentatore Step-down	OUT-	GND	-	Nero
Encoder 1	Segnale (centrale)	PB4 (D5)	-	Rosso
Encoder 1	Alimentazione (+)	5V	-	Rosso
Encoder 1	GND (-)	GND	-	Nero
Encoder 2	Segnale (centrale)	PB10 (D6)	-	Rosso
Encoder 2	Alimentazione (+)	5V	-	Rosso
Encoder 2	GND (-)	GND	-	Nero
Limit Switch 1	Segnale	PA8 (D7)	-	Rosso
Limit Switch 1	Comune	GND	-	Nero
Limit Switch 2	Segnale	PA9 (D8)	-	Rosso
Limit Switch 2	Comune	GND	-	Nero
CNC Shield	X.STEP	PA10 (D2)	X.STEP/DIR	Verde
CNC Shield	X.DIR	PB3 (D3)	X.STEP/DIR	Giallo
CNC Shield	Y.STEP	PB5 (D4)	Y.STEP/DIR	Verde
CNC Shield	Y.DIR	PB13 (D9)	Y.STEP/DIR	Giallo
CNC Shield	EN/GND	GND	EN/GND	Arancione
CNC Shield	Alimentazione	VIN	12~36V	Rosso/Nero
Stepper Motor X	Bobina A	-	X driver A1/A2	Rosso/Blu
Stepper Motor X	Bobina B	-	X driver B1/B2	Giallo/Viola
Stepper Motor Y	Bobina A	-	Y driver A1/A2	Rosso/Blu
Stepper Motor Y	Bobina B	-	Y driver B1/B2	Giallo/Viola

Tabella 5: Collegamenti firmware-manipolatore

Note

- Il Nucleo-F411RE è alimentato a 5V dal regolatore step-down (alimentatore DC/DC).
- I due encoder sono alimentati a 5V dalla scheda stessa (Nucleo).
- I due limit switch utilizzano una connessione GND comune per i segnali.
- La scheda CNC Shield è alimentata separatamente con tensione compresa tra 12~36 V, separata dalla tensione del microcontrollore.
- Sono collegati due motori stepper, uno sull'asse X e uno sull'asse Y, ciascuno con 4 fili (due coppie di bobine).

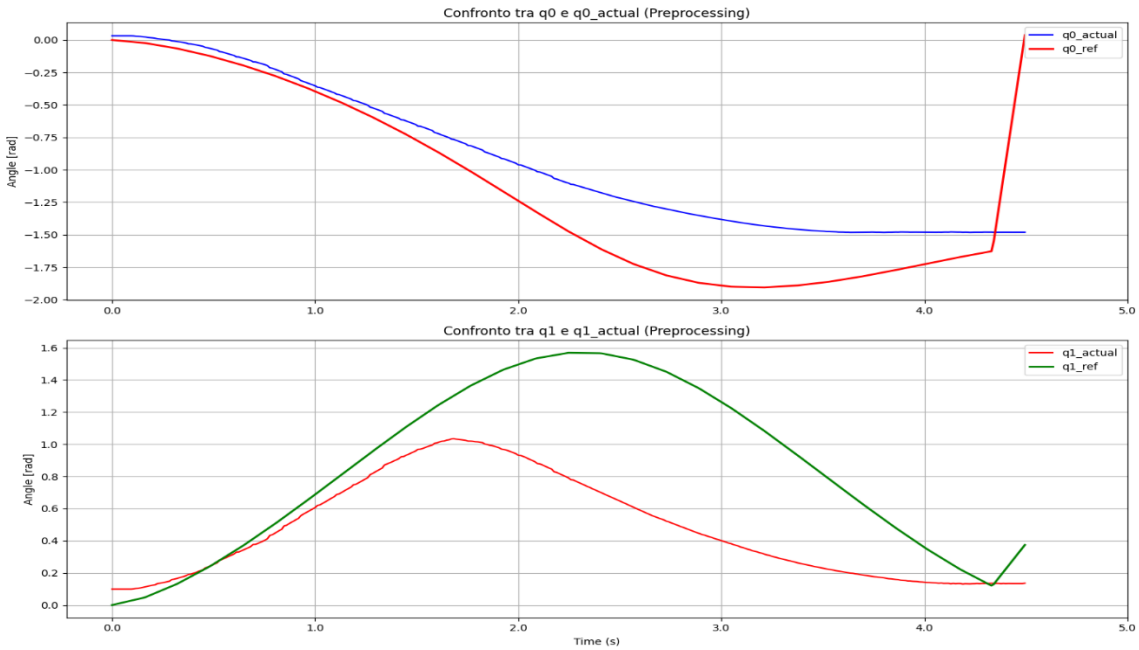
Test di laboratorio: valutazioni sul funzionamento

In questa sezione del report verranno descritte e commentate le varie esperienze di laboratorio effettuate per testare su banco il manipolatore. Dalle analisi preliminari svolte sui codici forniti e successivamente dai primi avvii del manipolatore, sono emerse alcune criticità. A valle di molteplici test, parte dei problemi sono stati parzialmente risolti, seppur restino alcuni open-issues da attenzionare in futuro e che verranno proposti al termine della trattazione.

FASE 1: Acquisizioni preliminari

Nel primo approccio alla valutazione delle criticità esistenti sul funzionamento del manipolatore, si è deciso di acquisire i dati provenienti dagli encoder nell'esecuzione della traiettoria predefinita (repeatable trajectory) per confrontarli con la traiettoria di riferimento, inviata in asincrono al firmware tramite interfaccia python.

Dai plot ottenuti, è evidente come l'inseguimento della traiettoria sia affetto da un errore sul transitorio più o meno ampio e che, la traiettoria di riferimento inviata lato front-end abbia un problema di interpolazione dei punti finali (picchi atipici negli ultimi campioni).



FASE 2: Setting dei parametri critici

A valle della prima esperienza di data-collection, il gruppo ha ritenuto opportuno agire sul codice, prima di effettuare nuovamente le misurazioni (proposte nel seguito). È stato infatti effettuato un ricalcolo dei registri del firmware, che utilizzavano un tempo di campionamento Tc errato e non corrispondente al tempo di campionamento con il quale venivano generate le traiettorie di riferimento lato frontend.

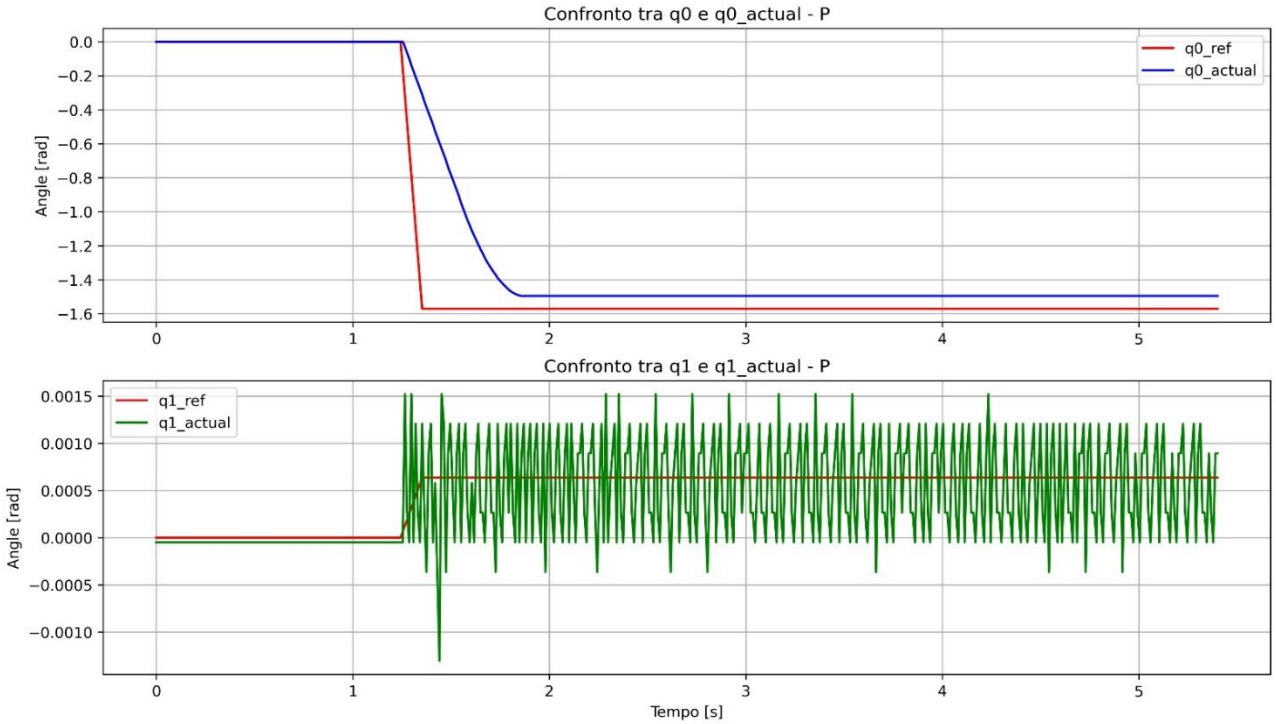
In seguito, è stato settato opportunamente il parametro che regola la velocità di scrittura dei ring-buffers del firmware (`datarate`), dal momento che, a seguito di diversi test, si è compreso come da questo dipendesse la capacità del manipolatore di eseguire traiettorie in maniera fluida e priva di movimenti *intermittenti*.

FASE 3: Ricalibrazione dei controllori

Le misurazioni effettuate in laboratorio hanno riguardato, in fase di test, l'utilizzo di un segnale a gradino e l'inseguimento di questo riferimento per la verifica delle prestazioni del controllo di posizione applicato (il codice sorgente prelevato in origine sembra impiegare controllori PID, ma da una attenta verifica degli script è emersa l'implementazione effettiva di un PI).

Avendo, in fase preliminare, notato tramite il plotting della repeatable trajectory una capacità scarsa di inseguimento, si è deciso di ricalibrare sul gradino i controllori.

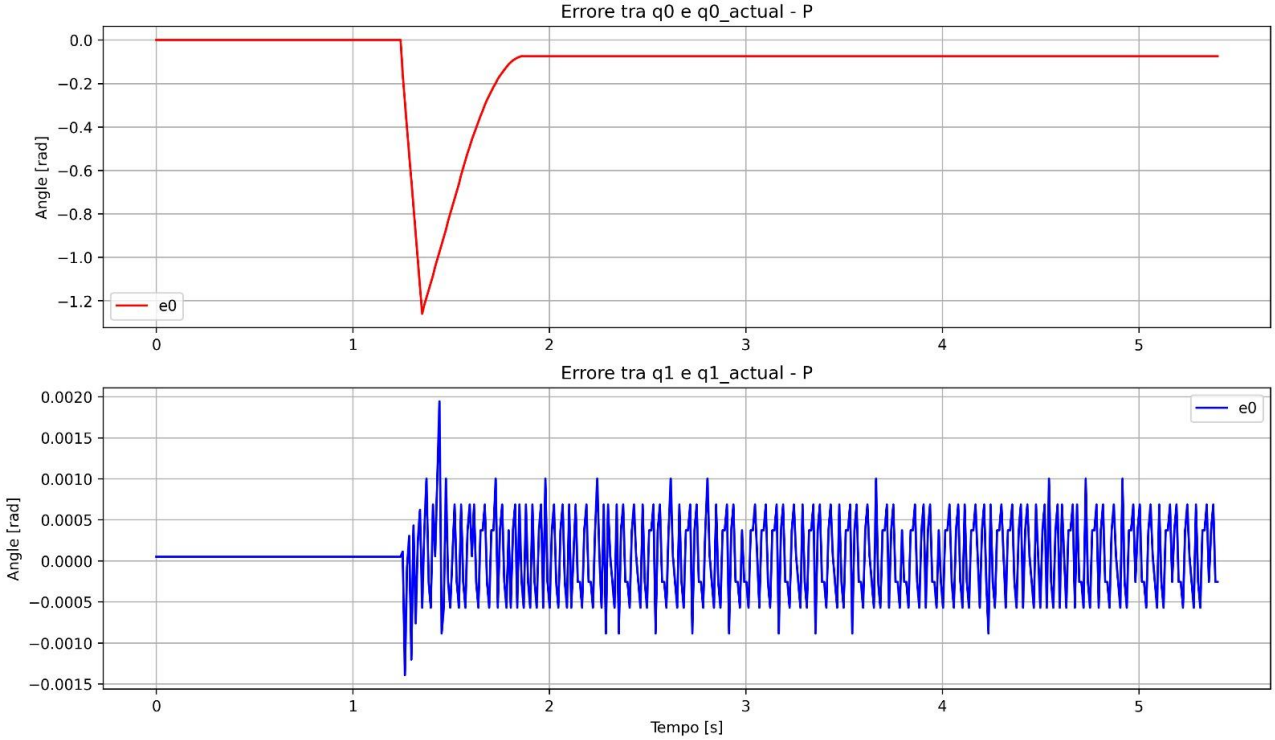
Procedendo per gradi, lo studio è cominciato con l'applicazione di un segnale a gradino in ingresso e l'utilizzo del solo contributo P del controllore, eliminando momentaneamente il contributo Integrale. Il grafico di inseguimento che ne risulta dall'applicazione di tale gradino, è il seguente:



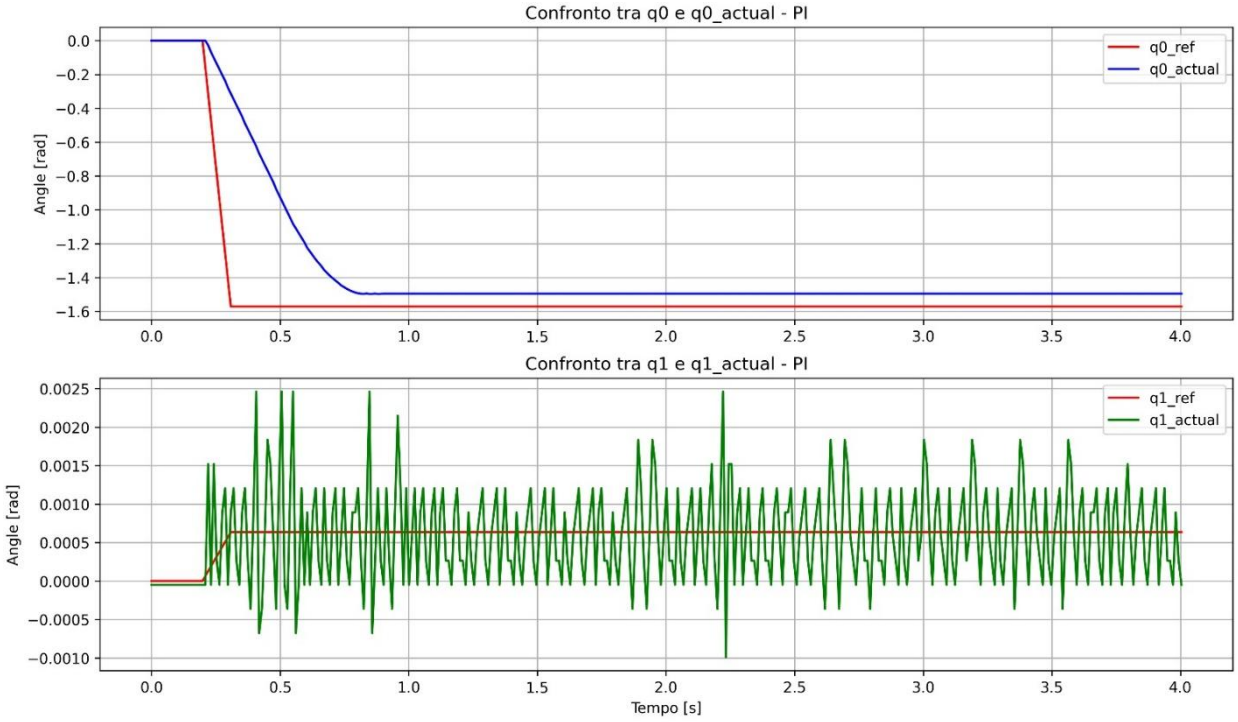
Da questo plot è evidente come la capacità di inseguimento del riferimento con l'utilizzo di un semplice controllore ad azione proporzionale opportunamente calibrato risulti nettamente migliore rispetto alla condizione di partenza, con un errore a regime costante. Il confronto fra *q1_ref* di riferimento e *q1_actual* è invece affetto da chattering, poiché il manipolatore entra in uno stadio di vibrazione continua attorno al punto di riferimento. È comunque interessante notare che le vibrazioni attorno al riferimento del secondo attuatore sono considerabili ampiamente trascurabili, essendo dell'ordine del centesimo di grado ($\approx \pm 0.02°$): potrebbe banalmente trattarsi di rumore sulle misurazioni.

D'altro canto, l'offset a regime nel caso del primo attuatore fa pensare alla possibilità di un controllo di tipo PI che verrà analizzato nel seguito.

A seguire il grafico degli errori di posizione retroazionati al controllore:

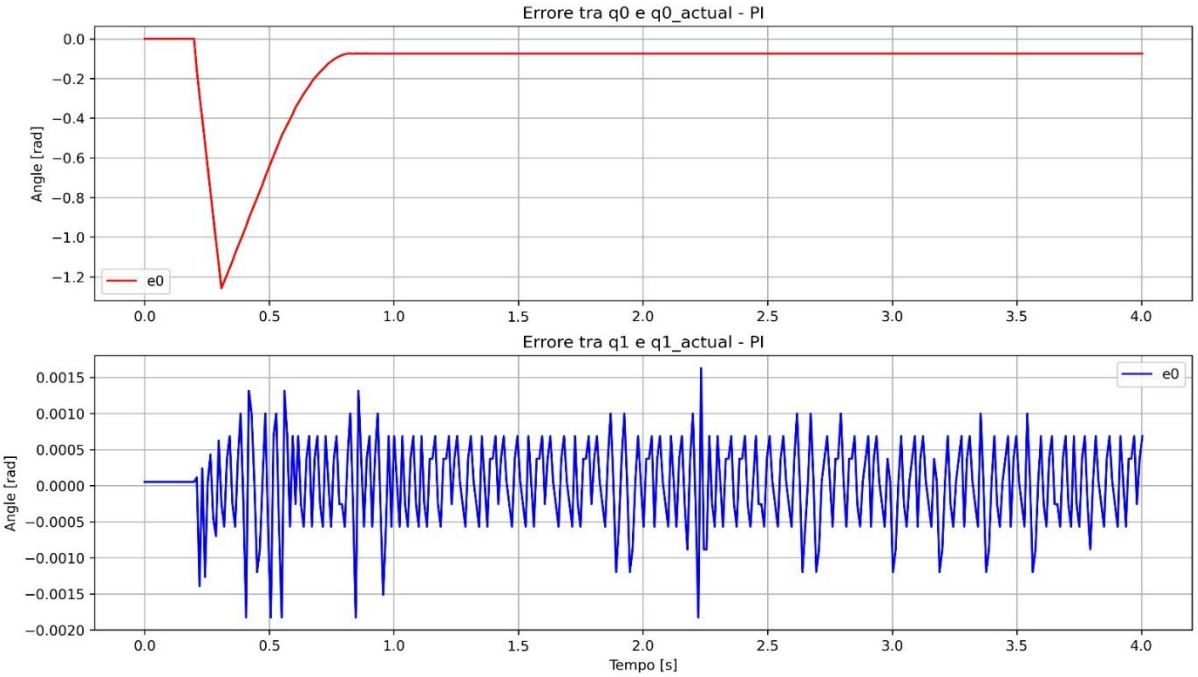


Successivamente, è stato analizzato il contributo dell'azione integrale, quindi è stato ridefinito il controllore PI (come in origine), mentre è stato mantenuto in ingresso un gradino di riferimento:



Il miglioramento dato dal contributo Integrale all'inseguimento della traiettoria a gradino è minimo, corrisponde a qualche centesimo nel caso del primo attuatore che vede comunque la permanenza di un offset a regime; mentre il fenomeno di chattering persiste se l'ingresso è un ingresso a gradino costante.

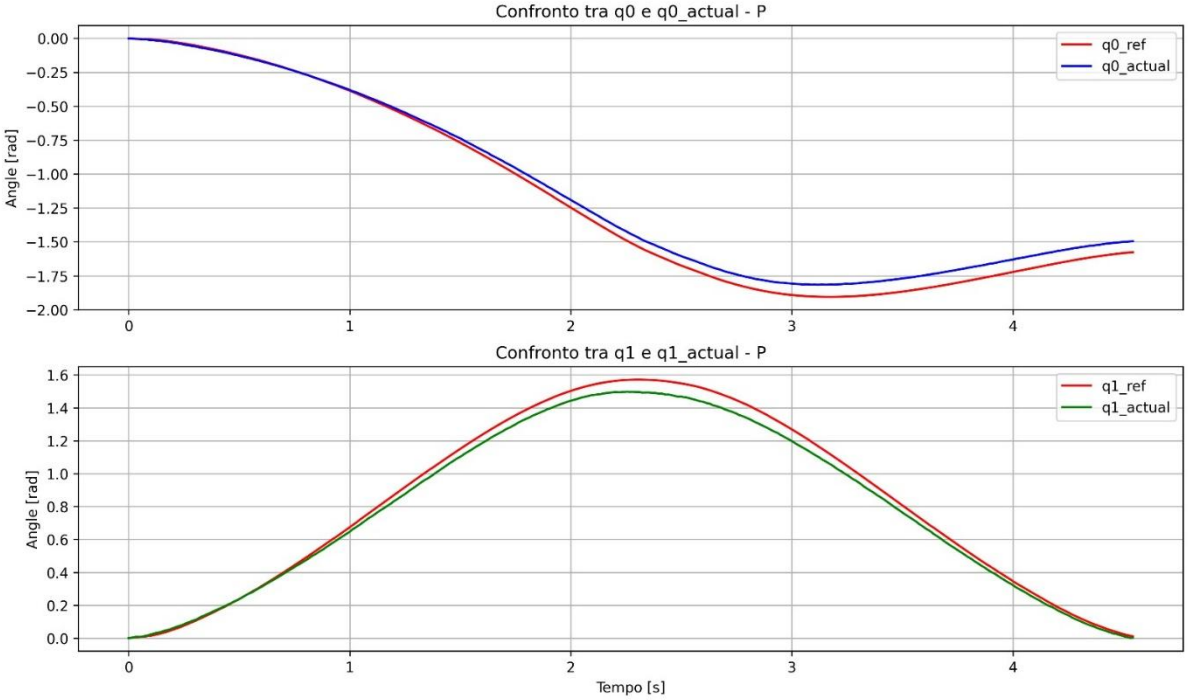
Di seguito è presente il grafico degli errori con l'utilizzo di un regolatore PI:



Facendo il confronto fra questo grafico degli errori e quello in cui utilizziamo un semplice regolatore P non emergono sostanziali miglioramenti delle prestazioni del manipolatore. L'offset a regime infatti resta comunque visibile, infatti né il primo né il secondo grafico degli errori convergono a zero, e questo si ripercuote sulla precisione di inseguimento del manipolatore di un segnale in ingresso, che può anche non essere un banale segnale a gradino.

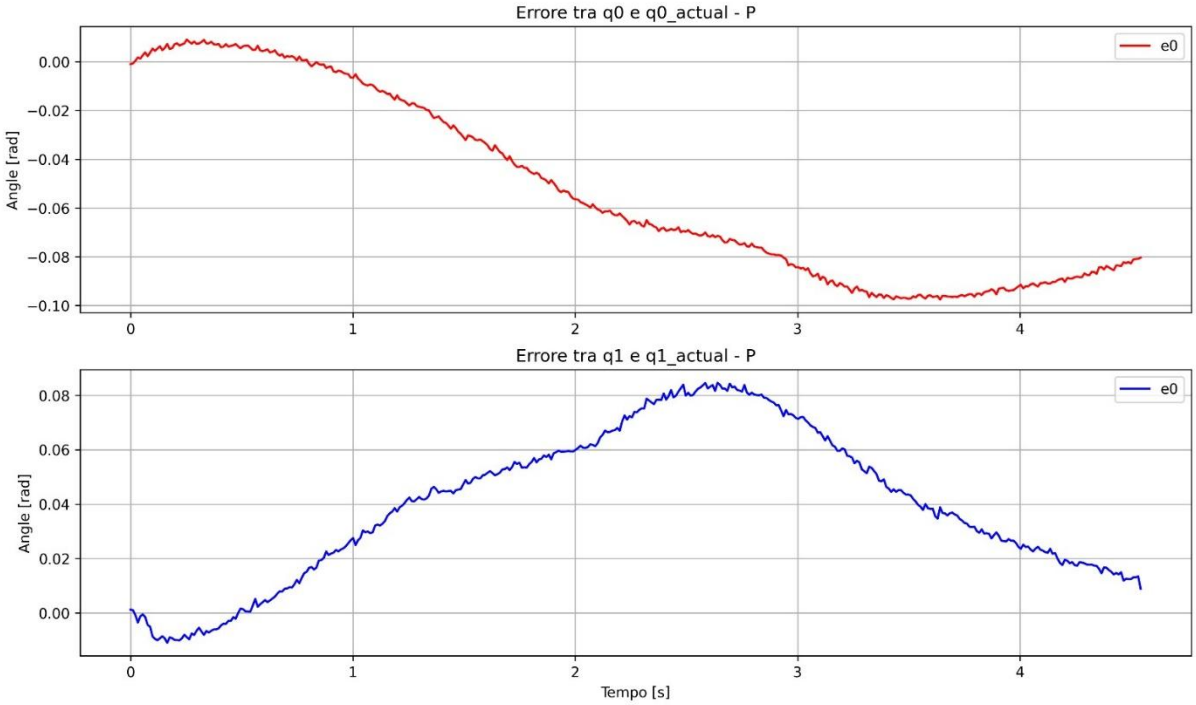
Per proseguire con l'esperienza di misurazione è stata, a questo punto, utilizzata una traiettoria più complessa da seguire. Questo, come si vedrà dai grafici successivi, ha completamente eliminato il fenomeno di chattering del manipolatore, ma ha evidenziato altre problematiche.

Di seguito viene mostrato il grafico dell'applicazione di una Repeatable Trajectory in ingresso, che consiste nella descrizione di una retta sul piano di lavoro (tracciata a partire dalla posizione iniziale, detta *Homing*), utilizzando solo il regolatore P:



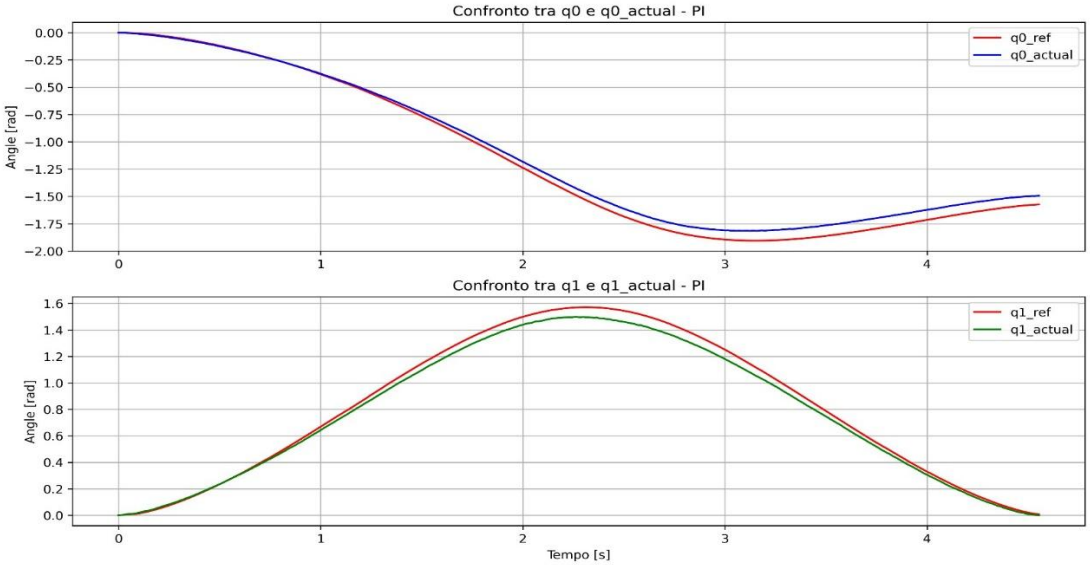
Da questo grafico emerge la scomparsa del fenomeno di chattering del manipolatore sul secondo attuatore, e conseguentemente un inseguimento più preciso della traiettoria.

Tuttavia, emerge anche un sostanziale aumento dell'errore su *q0* durante l'avanzamento della traiettoria e che sarà visibile dal successivo grafico degli errori:



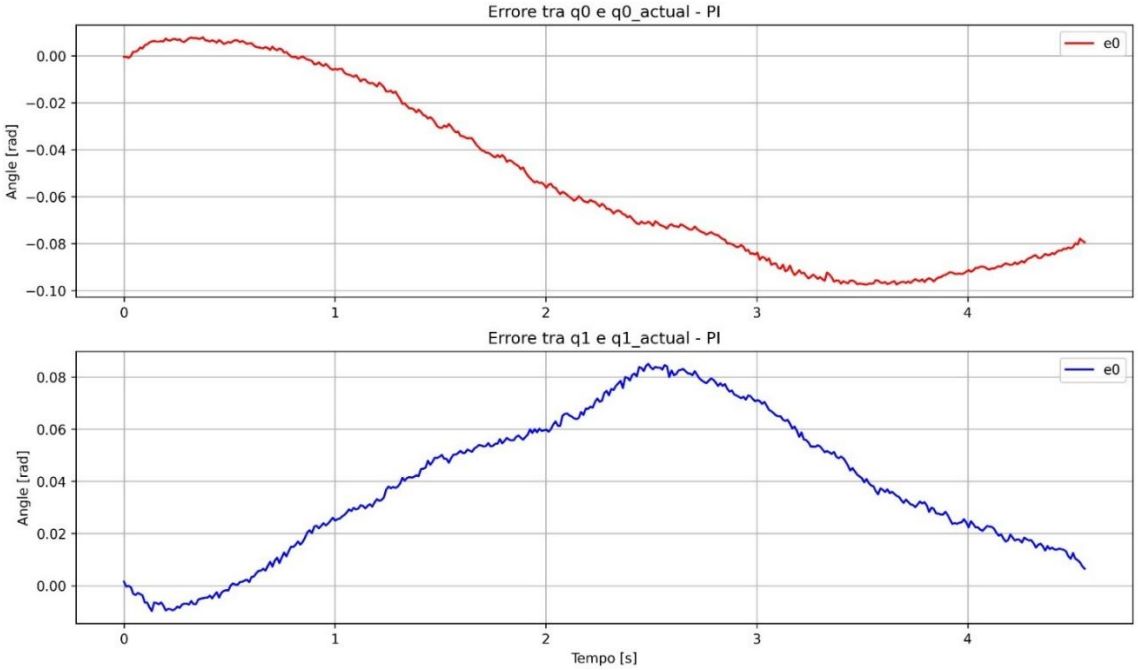
Mentre l'errore su *q1* tende ad annullarsi dopo un leggero aumento dovuto all'inseguimento dei punti del transitorio della Repeatable Trajectory, l'errore su *q0* non tende ad annullarsi, andando a definire un vero e proprio offset sulla traiettoria (come visto anche nel caso del gradino), visibile anche ad occhio nudo durante la simulazione della traiettoria stessa in laboratorio.

Per concludere le misurazioni in laboratorio è stata utilizzata la Repeatable Trajectory in ingresso, dopo aver effettuato nuovamente un Homing del manipolatore, utilizzando il regolatore PI, con il seguente risultato:



Da questi grafici si evince che il contributo del PI non è facilmente apprezzabile, e gli errori vengono mantenuti quasi costanti rispetto all'esperienza utilizzando il semplice controllore P.

Il grafico degli errori ricavato è mostrato di seguito:



Da questo grafico emerge un miglioramento sugli errori di inseguimento della traiettoria dell'ordine di 0.001 radianti. Ciò implica un effetto minimo di reiezione degli errori da parte del contributo Integrale dei PI.

Raccolta dati

Per raccogliere dati utili per effettuare valutazioni sul funzionamento del firmware, è stata introdotta una funzione di log di dati nel codice sorgente. Il codice è il seguente:

```c
void log_data_csv(char *prefix, man_t *manip) {
    float q0, q1, q0_actual, q1_actual;
    uint32_t timestamp;
    char buffer[256];
    // Retrieve the latest desired and actual values from ringbuffers
    rblast(&manip->q0, &q0);
    rblast(&manip->q1, &q1);
    rblast(&manip->q0_actual, &q0_actual);
    rblast(&manip->q1_actual, &q1_actual);
    // Get a timestamp in milliseconds
    timestamp = HAL_GetTick();
    // Format as a CSV string: q0,q1,q0_actual,q1_actual,timestamp\n

    printf("%s%f,%f,%f,%f,%lu\n", prefix, q0, q1, q0_actual, q1_actual, timestamp);
    fflush(stdout);
}
```

Questo codice definisce una funzione, `log_data_csv`, il cui scopo è estrarre e stampare in formato CSV una serie di dati relativi al manipolatore. La funzione riceve due parametri in input: un puntatore a carattere (*prefix*) che funge da prefisso per la stringa stampata, e un puntatore a una struttura di tipo *man_t* (*manip*) che contiene, tra gli altri dati, dei ringbuffer relativi alle variabili di controllo del manipolatore.

Dichiarazione delle variabili

- **Variabili float**: vengono dichiarate quattro variabili di tipo `float` (q0, q1, q0_actual, q1_actual). Queste rappresentano rispettivamente i valori di riferimento (q0 e q1) e i valori effettivi registrati dagli encoder e post-processati (q0_actual e q1_actual) delle due variabili di giunto associate ad ognuno degli attuatori del manipolatore.
- **Timestamp**: viene dichiarata una variabile `timestamp` di tipo `uint32_t`, usata per immagazzinare il tempo corrente in millisecondi.
- **Buffer di caratteri**: viene allocato un array di caratteri `buffer` di dimensione 256. In questo codice, però, il buffer non viene effettivamente utilizzato. La sua allocazione è pensata per un possibile utilizzo futuro: qualora si volesse rendere la comunicazione dei dati accessibile da front-end piuttosto che da terminale seriale, pensare di scrivere su un buffer i dati raccolti ad ogni step di questa funzione potrebbe rappresentare una strategia fruttuosa.

Estrazione dei dati dai ringbuffer

Il codice utilizza la funzione `rblast` per estrarre l'ultimo elemento dai ringbuffers contenuti nella struttura *manip*. Questi ringbuffer, istanziati nell'omonimo script firmware, tengono traccia di una serie di valori nel tempo, e la funzione `rblast` si occupa di recuperare l'ultimo dato disponibile:

- `rblast(&manip->q0, &q0);` estrae il valore di *q0*.
- `rblast(&manip->q1, &q1);` estrae il valore di *q1*.
- `rblast(&manip->q0_actual, &q0_actual);` estrae il valore misurato di *q0*.
- `rblast(&manip->q1_actual, &q1_actual);` estrae il valore misurato di *q1*.

Questa procedura permette di aggiornare le variabili locali con i dati più recenti registrati nei rispettivi buffer.

Ottenimento del timestamp

La funzione chiama `HAL_GetTick()`, una API dell'ambiente di sviluppo STM32, che restituisce il tempo trascorso (in millisecondi) dall'avvio del sistema. Il valore ottenuto viene salvato nella variabile `timestamp`. Questo consente di associare ad ogni riga di dati anche un'informazione temporale, utile per il plotting dei dati e la stima della durata complessiva dell'esecuzione del movimento..

Formattazione e stampa dei dati

La funzione utilizza `printf` per formattare e stampare i dati raccolti. La stringa di formato specifica che i dati saranno separati da virgole (csv format):

- Il prefisso (*prefix*) viene stampato per primo, seguito dai valori in virgola mobile (q0, q1, q0_actual, q1_actual) e, infine, dal timestamp. Il prefisso è stato introdotto nella striga da stampare con lo scopo di introdurre un flag identificativo qualora si volesse chiamare la funzione di log data contemporaneamente in più punti del codice: in questo caso è possibile scegliere un qualsiasi carattere. Se vien lanciata una sola volta all'interno del loop infinito, allora il campo può anche essere uno spazio vuoto.
- Il risultato finale è una riga in formato CSV che segue la struttura:

```
prefix q0,q1,q0_actual,q1_actual,timestamp\n
```

Infine, la chiamata a `fflush(stdout)` assicura che il buffer di output venga svuotato immediatamente, garantendo che la stringa formattata venga visualizzata senza ritardi.

Stampa a video

Dalla dashboard di STM32 (CubeIDE) è stata attivata la funzione di debug Serial Wire Viewer (SWV) per permettere la visualizzazione in tempo reale dati estratti dal manipolatore. In pratica, abilitando SWV si reindirizza l'output generato dalla funzione printf – che in questo caso stampa la stringa CSV descritta in precedenza – verso un canale di debug che può essere monitorato direttamente da un ambiente di sviluppo (nel caso in oggetto è stato stampato tutto su terminale).

La funzione `log_data_csv` utilizza `printf` per formattare e stampare i dati in formato CSV. In una configurazione tipica, l'output di `printf` viene indirizzato verso una porta seriale tradizionale. Tuttavia, poiché il canale seriale è impegnato dalla trasmissione dei dati dal front-end al firmware, non era possibile utilizzare, per come è stato progettato il firmware stesso, lo UART.

Abilitando SWV, i dati di log vengono visualizzati immediatamente sul debugger collegato. Questo metodo offre il vantaggio di non dover utilizzare una porta UART separata, liberando così risorse hardware e permettendo una diagnosi in tempo reale, ma resta in ogni caso una soluzione adatta a situazioni di testing. Sarebbe opportuno stabilire una connessione seriale dedicata al log dei dati così da esser in grado di accedervi indipendentemente dalla debug mode. .

Considerazioni finali e sviluppi futuri

Sulla base delle misurazioni effettuate e confrontandole con le prime si evince (eseguite sul manipolatore con il codice generato dal gruppo di lavoro precedente) che c'è stata un significativo aumento delle prestazioni del manipolatore, frutto dei ricalcoli effettuati su tempo di campionamento, datarate, e costanti Kp e Ki.

Tuttavia, l'offset su *q0* non sembra essere dovuto all'anello di controllo e può essere attribuito a diverse cause, da avvalorare singolarmente:

- Cattivo funzionamento dei Limit Switch, che talvolta tendevano a bloccare il manipolatore durante la procedura di Homing;
- Gestione approssimativa della procedura di Homing, che genera un offset costante sul riferimento del primo attuatore;
- Ricalibrazione degli encoder incrementali presenti.

Se queste verifiche non portano risultati, si può considerare l'ipotesi di utilizzo di un controllore PID invece che un PI, aggiungendo in fase di controllo un'azione Derivativa per il miglioramento delle prestazioni, quantomeno a regime.

Sulla base di tutte queste considerazioni e analisi precedenti, per proseguire il lavoro si potrebbe:

- Effettuare una revisione della procedura di Homing del manipolatore, gestita non più come un ritorno a due punti stimati empiricamente e passati al manipolatore dopo che esso ha triggerato i due limit switch, ma come una traiettoria di più punti che permetta al manipolatore di tornare alla posizione neutra, con o senza l'ausilio dei Limit Switch. Il parametro da tenere in considerazione inizialmente è `pos_real`, presente nel file `custom.c`;
- Revisione del funzionamento hardware dei due Limit Switch e dei due encoder incrementali, per migliorare anche la procedura di Homing: durante le prove di laboratorio sono stati frequenti gli episodi di non funzionamento dei limit switch (anche se correttamente alimentati come verificato in tali occorrenze con multimetro).
- Revisione del controllo PI e passaggio a controllori di tipo PID, con conseguente verifica del miglioramento o peggioramento delle prestazioni complessive di inseguimento di traiettorie in ingresso, a gradino o più complesse.
- Revisione delle logiche di comunicazione con il firmware: valutare l'opportunità di stabilire una comunicazione sincrona dei riferimenti potrebbe risolvere alcuni problemi di inseguimento e migliorerebbe sicuramente il processo di raccolta dei dati, che attualmente avviene su di un unico bus RX/TX e dipende dal interamente dal frontend. Eliminare questa dipendenza garantirebbe maggiore scalabilità dei vari livelli di programmazione (software e firmware/middleware).